netimpute: Joint Multiple Imputation of Node Attributes and Network Ties in R

Jane Doe
Department of Sociology, University Example

Abstract

Missing data in social network studies routinely affects both node-level attributes (e.g. survey non-response) and the network ties themselves (e.g. un-reciprocated nominations, boundary specification problems, or partially observed rosters). Standard multiple imputation software such as **mice** (van Buuren and Groothuis-Oudshoorn, 2011) handles the former well but has no notion of network structure, while dedicated network-imputation procedures typically treat tie imputation and attribute imputation as separate problems. This paper introduces **netimpute**, an R package that (i) computes a broad battery of node-level structural and attribute-scale-appropriate homophily measures across one or more networks, (ii) reduces these to a manageable set of principal-component predictors while optionally preserving specific raw measures that are themselves part of a substantive hypothesis, (iii) provides a dyadic (cell-level) regression for network ties in the spirit of MR-QAP (Krackhardt, 1988) but fitted directly rather than by permutation, and (iv) combines both into `netmice()`, a chained-equations routine that jointly imputes missing node attributes and missing network ties by cycling between them, using network-derived predictors for attributes and attribute-derived (and other-network-derived) predictors for ties. We describe the package's design choices, the measures it computes for binary and non-negative weighted networks, and illustrate its use with a reproducible example.

Keywords: multiple imputation, missing data, social network analysis, R, predictive mean matching, dyadic regression, MR-QAP.

1 Introduction

Two kinds of missingness arise in network studies, and they are usually handled by entirely different tools. Missing *node attributes* (age, performance, survey responses) are the domain of general-purpose multiple imputation (MI) software, most prominently **mice** (van Buuren and Groothuis-Oudshoorn, 2011), which implements the fully conditional specification / chained equations approach to MI (Rubin, 1987; Little and Rubin, 2002). Missing *network ties* — arising from partial rosters, non-response to a name-generator question, or top-coded nomination limits — are instead the subject of a smaller, more specialized literature on network imputation (Huisman, 2009; Krause et al., 2020), often developed alongside exponential random graph models (Robins et al., 2007) rather than as an extension of standard MI.

In practice, however, the two kinds of missingness are rarely independent problems. A respondent who did not answer the attribute survey is often the same respondent whose outgoing nominations are also missing; and the node-level measures researchers use to *explain* attributes (e.g. is a central employee also a high performer?) are themselves undefined until the network is complete, while the covariates used to predict a missing tie (e.g. do two employees in the same department reciprocate ties more often?) are undefined until the attributes are complete. Imputing one without regard to the other risks two failure modes: treating an incomplete network as if it were complete when computing centrality-type predictors for attribute imputation, and

treating incomplete attributes as if observed when building the dyadic covariates used to impute ties.

netimpute addresses this by keeping node-level measures, dyadic regression, and the imputation loop itself in one package built around a single, consistent representation of the two data types: a node attribute data frame and a (possibly weighted) adjacency matrix. Networks with unknown ties are represented as adjacency matrices with `NA` marking the unknown cells, exactly parallel to how `NA` marks a missing attribute value, so that the same chained-equations logic that underlies **mice** extends naturally to also cycle through networks.

The remainder of this paper is organised as follows. Section 2 briefly situates **netimpute** relative to existing R tools for network analysis and for missing data. Section 3 describes the package's four building blocks: node-level measures (`net_measures_core()`, `net_measures_full()`), predictor construction for imputation (`net_predictors()`), dyadic regression (`dyad_regression()`), and the joint imputation routine (`netmice()`). Section 4 works through a reproducible example. Section 5 discusses current limitations — most notably that signed networks are not yet supported — and directions for future work.

2 Related work

R's network analysis ecosystem is mature: **igraph** (Csardi and Nepusz, 2006) and **sna** (Butts, 2008) both provide extensive node- and network-level measures, and **netimpute** builds directly on **igraph** for its graph representation and uses **sna** as an optional source of a handful of additional centrality measures (Gil-Schmidt power, flow/load/stress centrality, information centrality, and prestige) that **igraph** does not provide. Neither package, however, is designed for missing data: both assume a fully observed network.

On the imputation side, **mice** (van Buuren and Groothuis-Oudshoorn, 2011) is the standard R implementation of chained-equations MI and is used internally by **netimpute** as the workhorse for the actual univariate imputation step (predictive mean matching for numeric and binary targets; multinomial logistic regression via `mice::mice.impute.polyreg()`, which in turn relies on **nnet** (Venables and Ripley, 2002), for nominal attributes with more than two categories). **netimpute** does not reimplement this machinery; it instead builds the network-aware predictor matrices that get handed to it, and generalises **mice**'s single chained-equations loop over attributes to a single loop over both attributes and networks.

Prior work on missing network data (Huisman, 2009; Krause et al., 2020) has established that simple treatments (listwise deletion, treating a missing tie as absent) can materially bias network-level statistics, and has proposed reconstruction procedures based on the observed structure. These procedures generally do not draw on node attributes as predictors, and are not implemented as a general-purpose, formula-extensible R package; **netimpute**'s dyadic regression step can be seen as a simple, non-permutation-based point-prediction analogue of MR-QAP (Krackhardt, 1988) used specifically to build a predictive (rather than inferential) model for tie imputation.

3 The netimpute package

3.1 Node-level measures

`net_measures_core(net, attributes, attr_types = NULL, id_col = NULL)` computes nine structural measures per node — out-degree, in-degree, a reciprocity measure, Bonacich power centrality (with an automatically rescaled exponent so it converges regardless of the network's spectral radius), normalised betweenness, an isolate indicator, Burt's constraint (Burt, 1992), harmonic closeness, and local clustering — together with, for every column of `attributes`, a scale-appropriate homophily/alter-similarity block: an E-I index, Blau's index,

and the modal alter category for binary or multinomial attributes; average absolute difference on outgoing and on incoming ties, plus the mean, minimum, and maximum alter value, for continuous attributes. `net_measures_full()` extends this to roughly 28 structural measures (the full degree family, eigenvector/alpha/PageRank centrality, structural holes measures, coreness, and, if **sna** is installed, its five additional measures) using exactly the same homophily block.

Only binary (0/1) and non-negative weighted networks are currently supported; supplying a network with a negative tie value raises an informative error rather than silently producing an ill-defined result (Section 5). For non-negative weighted networks, two measures are redefined rather than computed on a nonsensical binarisation: reciprocity becomes the average $|w_{ij} - w_{ji}|$ over alters tied in either direction (in the original weight units, with *lower* values now indicating *more* reciprocal ties, the opposite direction from the binary $[0, 1]$ ratio), and local clustering becomes the geometric-mean weighted clustering coefficient of Opsahl and Panzarasa (2009): for node i and each pair of its neighbours (j, k), the two-path value $\sqrt{w_{ij} w_{ik}}$ is summed over every such pair to form a denominator, and over only the pairs where the triangle is actually closed (j and k are themselves tied) to form the numerator. This ratio reduces exactly to the ordinary binary clustering coefficient when all weights equal 1.

3.2 Predictors for imputation

`net_predictors()` applies `net_measures_core()` or `net_measures_full()` across a list of networks (and a matching list of attribute data frames sharing the same schema), stacks the results, and offers three output modes. `output = "measures"` returns the stacked raw measures. `output = "pca"` instead returns the first `n_components` principal components of the (mean-imputed, centred/scaled) measure matrix, addressing the multicollinearity that is otherwise inevitable once a dozen or more structural and homophily measures are computed per network. `output = "both"` returns the principal components *and* a user-specified set of raw measures (`keep_vars`), preserved untransformed; this is for the situation where a specific measure — degree, say — is itself part of a hypothesised relationship the analyst does not want diluted by folding it into a composite component, optionally with the remaining predictors first residualised on `keep_vars` (`residualize_kept = TRUE`) so the components capture variance orthogonal to it.

3.3 Dyadic regression

`dyad_regression()` vectorises a target network's adjacency matrix (excluding the diagonal) and regresses it on: for every node attribute, an ego value, an alter value, and either an absolute difference (continuous attributes) or a same-category indicator (categorical attributes) — the absolute difference is used rather than the signed difference because the latter is an exact linear combination of the ego and alter columns already in the model and would make the design matrix singular; reciprocity (the transposed target network); the log number of two-paths; and, for every *other* network supplied, its tie value, its reciprocity (transpose), the sender's out-degree, and the receiver's in-degree in that network. When many other networks are supplied, `other_net_predictors = "pca"` replaces these terms with the first `n_components` principal components instead. The model is fit by `glm()` with a user-specified `family` (default Gaussian) on dyads with an observed tie value; dyads with an unknown value (`NA`) are left out of the fit but can be predicted from it. This is the same predictor set used by MR-QAP (Krackhardt, 1988), fit directly by maximum likelihood rather than by permutation, since the goal here is prediction for imputation rather than a permutation-based significance test.

3.4 Joint imputation

`netmice(data, net_list, m = 5, maxit = 20, method = "pmm", donors = 5, ...)` is a chained-equations routine, architecturally modelled on **mice**'s own sampler, generalised to cycle through both attributes and networks. Within one iteration, every attribute and every network with missing values is visited exactly once, in a single interleaved sequence whose order is randomised independently for each of the m imputation chains. Visiting an attribute rebuilds its network-derived predictors (Section 3.1) from whichever networks were most recently updated; visiting a network rebuilds its dyadic predictors (Section 3.3) from whichever attributes and other networks were most recently updated. Because these predictors depend on *the other* data type, they are never cached: a variable that depends on both a network and an attribute is effectively recomputed at both points in the cycle.

The actual univariate draw is delegated to **mice**: predictive mean matching (`mice.impute.pmm()`) for numeric and binary targets (a binary attribute is integer-coded, imputed as a linear-probability PMM problem, and mapped back to its two labels — harmless since PMM always returns an observed donor value regardless of the working model's exact functional form), and multinomial logistic regression (`mice.impute.polyreg()`) for nominal attributes with more than two categories, avoiding the alternative of imposing an arbitrary numeric ordering on the categories. Only PMM is currently exposed as a selectable `method`; the interface is deliberately left open (`method` is a plain string dispatched through an internal switch) for additional methods to be added without changing the call signature.

Because the auto-generated predictor set can plausibly approach or exceed the number of observed cases available for a given target — several networks' worth of structural and homophily measures adds up quickly relative to a modest number of nodes — `netmice()` applies the same fix offered explicitly through `net_predictors(output = "pca")` automatically inside the loop: the auto-generated predictors are capped at $\max(5, \lfloor n_{\text{observed}}/3 \rfloor)$ columns, reducing to that many principal components when exceeded. This does not apply to predictors a user has explicitly requested via `models` (see below), which are treated as a deliberate choice.

A specific, hypothesised relationship can be protected from dilution or omission via the `models` argument, a list of formula strings whose left-hand side names the attribute or network they apply to; the right-hand side is evaluated with `model.matrix()` (so interaction syntax such as `performance * gender` works) and its columns are *appended to*, never substituted for, the auto-generated predictors. A network's own formula must reference the dyadic predictor names from Section 3.3 (e.g. `age_absdiff`, `advice_tie`), not raw node attributes.

For `ncores` > 1, the m chains run as independent **future** futures (`future::multisession`), each with its own random visit order and an explicitly pinned random-number generator so that results are identical whether run sequentially or in parallel given the same `seed` (multisession workers otherwise default to a different RNG algorithm for proper parallel streams, which would silently change results if the seed value were reused as-is). `netmice()` returns an object of class `netmids` recording, in parallel to **mice**'s own convergence diagnostics, the mean and variance of each imputed (not observed) attribute value across iterations, and, for every network, its density, reciprocity, transitivity, isolate count, and average inverse geodesic distance (global efficiency) at every iteration — computed as the mean of $1/d(i,j)$ over all ordered pairs, which is finite by construction even in the presence of isolates or disconnected components, since $1/\infty = 0$ rather than undefined.

4 Illustrative example

The example below simulates a 40-node organisation with two networks (a binary friendship network and a Poisson-weighted advice network) and four attributes spanning all three supported scales (continuous, binary, multinomial). Numbers below are the actual output of running this

code with R 4.6.0 and **netimpute** 0.1.0.

```
set.seed(20260701)
n <- 40
friends <- matrix(rbinom(n * n, 1, 0.08), n, n); diag(friends) <- 0
advice  <- matrix(rpois(n * n, 0.35), n, n);     diag(advice)  <- 0

attrs <- data.frame(
  age         = round(rnorm(n, 34, 7)),
  gender      = sample(c("F", "M"), n, replace = TRUE),
  department  = sample(c("sales", "eng", "hr"), n, replace = TRUE,
                       prob = c(0.4, 0.4, 0.2)),
  performance = round(rnorm(n, 50, 10), 1)
)

core_out <- net_measures_core(friends, attrs)
head(core_out[c("node_id", "outdegree", "indegree",
                "reciprocity_ratio", "betweenness", "constraint")])
```

```
  node_id outdegree indegree reciprocity_ratio betweenness constraint
1       1         5        4             0.125       0.095      0.154
2       2         3        1             0.000       0.047      0.302
3       3         2        1             0.000       0.005      0.333
4       4         3        3             0.000       0.062      0.187
5       5         1        0             0.000       0.000      1.000
6       6         2        3             0.000       0.048      0.200
```

A dyadic regression of friendship ties on the node attributes plus the advice network illustrates the predictor set from Section 3.3 (selected coefficients shown; this network was simulated with no true structure, so none of these are significant, as expected):

```
dr <- dyad_regression(list(friends = friends, advice = advice),
                      attrs, target = "friends")
summary(dr$model)$coefficients[
  c("age_absdiff", "department_same", "advice_tie"), ]
```

```
                    Estimate  Std. Error     t value  Pr(>|t|)
age_absdiff      2.592629e-05 0.001013589  0.02557869 0.9795967
department_same -8.697058e-03 0.014094315 -0.61706140 0.5372855
advice_tie      -1.102619e-03 0.011362338 -0.09704159 0.9227060
```

We now introduce missingness in three attributes and in five percent of the friendship network's off-diagonal cells, and impute both jointly, using `models` to protect a hypothesised interaction between age and department in predicting performance:

```
attrs_miss <- attrs
attrs_miss$age[sample(n, 4)]         <- NA
attrs_miss$performance[sample(n, 5)] <- NA
attrs_miss$department[sample(n, 3)]  <- NA

friends_miss <- friends
off <- which(row(friends_miss) != col(friends_miss))
friends_miss[sample(off, round(length(off) * 0.05))] <- NA

## maxit = 5 (below the default of 20) keeps the tables below compact.
fit <- netmice(
  attrs_miss, list(friends = friends_miss, advice = advice),
  m = 5, maxit = 5, donors = 5, seed = 20260701,
  models = list("performance ~ friends_indegree + age * department")
```

```
)
fit
```

```
Class: netmids
Imputations (m): 5   Iterations (maxit): 5   Method: pmm   Donors: 5
   Cores: 1
Variables with missingness: age, department, performance
Networks with missingness: friends
Custom models for: performance
```

A completed data set is extracted with `complete_netmice()`, and convergence can be inspected via the chain means for each imputed attribute across iterations, alongside network-level diagnostics tracked at every iteration for every network:

```
completed <- complete_netmice(fit, 1)
anyNA(completed$data)
anyNA(completed$net_list$friends[row(friends) != col(friends)])
round(fit$chainMean["age", , ], 2)
round(t(fit$netChain["friends", , , 1]), 3)
```

```
[1] FALSE
[1] FALSE
      1     2     3     4  5
1 38.25 30.50 29.50 33.75 33
2 33.50 22.25 29.50 34.25 37
3 40.00 33.25 29.25 38.00 29
4 30.00 27.50 32.50 37.50 25
5 36.00 37.75 30.00 31.50 36

  density reciprocity transitivity n_isolates avg_inv_geodesic
1   0.069       0.056        0.122          0            0.492
2   0.070       0.037        0.127          0            0.500
3   0.070       0.037        0.123          0            0.497
4   0.070       0.037        0.144          0            0.496
5   0.071       0.036        0.139          0            0.497
```

Both attributes and network ties are complete in the returned object, and the tracked network diagnostics are stable across iterations, consistent with (though of course not a substitute for a formal test of) convergence.

5 Discussion and future work

Signed networks are not yet supported. Several of the measures in Section 3.1 are path-based (betweenness, closeness) and require non-negative edge costs, while others (the weighted reciprocity and clustering formulas above) assume weights are magnitudes rather than signed valences; **netimpute** currently rejects any network with a negative tie value with an explicit error rather than silently producing an ill-defined result. Extending the package to signed networks would require, at minimum, separate treatment of positive and negative sub-graphs for the path-based measures and a structural-balance-aware notion of reciprocity and clustering, which we leave to future work.

Predictor dimensionality. The automatic PCA-based cap described in Section 3.4 prevents the imputation model from crashing on an exactly rank-deficient design, but a more principled treatment — e.g. allowing the user to specify `net_predictors()`-style `keep_vars` directly within `netmice()`, or exposing the ridge parameter passed to **mice** — would give more control over exactly which information is retained versus compressed.

Additional imputation methods. `netmice()`'s `method` argument is deliberately restricted to predictive mean matching for now, dispatched through a single internal switch so that alternative methods (e.g. classification and regression trees, or random forests, both already available as **mice** univariate imputation methods) can be added without changing the function's interface.

6 Summary

netimpute provides a single, coherent R interface for a problem that is otherwise split across separate tools: computing structural and homophily measures on networks with mixed attribute types and mixed (binary or non-negative weighted) tie types, reducing those measures to a workable predictor set for imputation, fitting a predictive dyadic model for network ties, and jointly imputing missing attributes and missing ties through a single chained-equations loop. The package, source code, and this manuscript's reproducible example script are available in the package's `paper/` directory.

References

Burt, R. S. (1992). *Structural Holes: The Social Structure of Competition.* Harvard University Press, Cambridge, MA.

Butts, C. T. (2008). Social network analysis with sna. *Journal of Statistical Software*, 24(6):1–51.

Csardi, G. and Nepusz, T. (2006). The igraph software package for complex network research. In *InterJournal, Complex Systems*, page 1695.

Huisman, M. (2009). Imputation of missing network data: Some simple procedures. *Journal of Social Structure*, 10(1):1–29.

Krackhardt, D. (1988). Predicting with networks: Nonparametric multiple regression analysis of dyadic data. *Social Networks*, 10(4):359–381.

Krause, R. W., Huisman, M., Steglich, C., and Snijders, T. A. B. (2020). Missing data in cross-sectional networks – An extensive comparison of missing data treatment methods. *Social Networks*, 62:99–112.

Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data.* Wiley, Hoboken, NJ, 2nd edition.

Opsahl, T. and Panzarasa, P. (2009). Clustering in weighted networks. *Social Networks*, 31(2):155–163.

Robins, G., Pattison, P., Kalish, Y., and Lusher, D. (2007). An introduction to exponential random graph (p*) models for social networks. *Social Networks*, 29(2):173–191.

Rubin, D. B. (1987). *Multiple Imputation for Nonresponse in Surveys.* Wiley, New York.

van Buuren, S. and Groothuis-Oudshoorn, K. (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, 45(3):1–67.

Venables, W. N. and Ripley, B. D. (2002). *Modern Applied Statistics with S.* Springer, New York, 4th edition.